

StackLite Hybrid QA Report

1. Goal and Corpus

The project builds a question-answering assistant for technical AI and data-science questions. The provided course dataset archive contains two Stack Exchange JSON files: *top_datascience_questions.json* and *top_ai_questions.json*. Each record includes a title, HTML body, tags, score, question ID, and source URL. The extracted corpus used in the local run contains 1,500 question records. The implementation converts each question into one searchable passage and assigns a stable document ID such as `datascience:12321` or `ai:10623`.

The system supports four user-facing requirements: index the document set, retrieve relevant passages, generate grounded answers with citations, and provide an interactive Gradio interface for live testing.

2. Methods

Preprocessing

HTML question bodies are converted to plain text with BeautifulSoup. Titles are HTML-unescaped, tags are preserved, and the searchable text concatenates title, tags, and body. Keeping tags in the index improves matches for technical vocabulary such as `keras`, `transformer`, `reinforcement-learning`, and `scikit-learn`.

The corpus loader deliberately keeps the original Stack Exchange URL and question ID attached to every document, since the final system is a cited QA assistant. A stable ID such as `ai:4456` lets the retriever, evaluator, UI, and generated answer all refer to the same evidence object. Preprocessing is intentionally light: no stemming, lemmatization, or aggressive stopwords removal is applied, as technical corpora often contain meaningful short tokens, version-like strings, and library identifiers that can be damaged by generic normalization.

BM25 Retrieval

The lexical retriever uses `rank-bm25` with a lightweight regex tokenizer. BM25 is strong for exact technical terms, library names, error names, and abbreviations. The notebook includes a top-10 BM25 inspection cell for sample questions.

BM25 is also the fastest index in the project — it is built immediately when the Gradio app starts, allowing answers in BM25 mode without downloading an embedding model. This is useful for demos on constrained machines or for students who only want to verify the basic retrieval milestone first.

Dense Retrieval

Semantic retrieval uses `sentence-transformers/all-MiniLM-L6-v2`. Document embeddings are normalized and query-document similarity is computed with cosine similarity via a dot product. Dense retrieval helps with paraphrases where the user does not repeat the exact Stack Exchange title.

MiniLM was selected because it is small enough for Google Colab free GPUs while still being a practical general-purpose sentence embedding model. The implementation embeds the 1,500 passages in batches and keeps the matrix in memory. For a larger Stack Exchange dump, the same interface could be backed by FAISS or another vector index without changing the UI contract.

Fusion

Hybrid retrieval uses Reciprocal Rank Fusion (RRF). For each candidate document, the fused score is computed as:

$$\text{score}(d) = \text{sum}(1 / (k + \text{rank}_i(d))) \quad \text{with } k = 60$$

RRF is robust because it does not require BM25 and cosine scores to share a numeric scale. The system retrieves a larger candidate pool from each retriever and then fuses the rankings. This preserves documents that are very strong in one retriever but only moderate in the other. The tradeoff is that hybrid mode must build dense embeddings before answering, so the first hybrid query is slower than BM25-only mode.

RAG Generation

The RAG layer formats top retrieved passages with title, tags, URL, passage text, and document IDs. It supports three backends:

Backend	Description
HuggingFace FLAN-T5 (backend="hf")	Open-source generation, no API key required.
OpenAI chat completions (backend="openai")	Used when OPENAI_API_KEY is configured.
Extractive fallback (backend="extractive")	For no-key demos and quick smoke tests; selects high-overlap sentences from retrieved passages.

Generated answers are instructed to cite source IDs such as [datascience:51065], and the final answer appends the source title and URL list. The extractive fallback keeps the demo usable without API keys and makes the citation pipeline testable in any environment.

3. Evaluation

The evaluation file contains 10 paraphrased questions with explicit relevant document IDs — a sanity-check set for the supplied corpus, not a large benchmark. The provided script reports MAP@10, MRR@10, nDCG@10, and HitRate@10 for BM25, MiniLM dense retrieval, and hybrid RRF.

The evaluation set covers both data-science and AI topics: scikit-learn preprocessing, Keras class weighting, categorical encoding, transformer positional encoding, model-based reinforcement learning, self-supervised learning, conversational context in ChatGPT, and GPT temperature.

Run evaluation:

```
python3 scripts/evaluate.py
```

For a fast lexical-only smoke test:

```
python3 scripts/evaluate.py --skip-dense
```

Metrics are saved to `reports/evaluation_results.json`. The local run on the 10-question evaluation set produced:

Method	MAP@10	MRR@10	nDCG@10	HitRate@10
BM25	0.800	0.800	0.849	1.000
MiniLM dense	0.933	0.933	0.950	1.000
Hybrid RRF	0.825	0.825	0.869	1.000

BM25 was already strong because many evaluation questions contain exact technical terms. MiniLM performed best on this curated paraphrase set. Hybrid RRF still retrieved every relevant document in the top 10 and provides a robust combined mode for live demos, especially when exact-match and semantic signals disagree.

The fact that MiniLM beats hybrid on this small evaluation set does not mean hybrid retrieval is unnecessary. On queries with exact library or API names, BM25 may rank the best source higher. RRF gives the application a conservative default that benefits from both signals without requiring a tuned score calibration step.

4. Citation Quality

Citation quality is checked with 10 evaluation questions and examples in `docs/citation_quality.md`. A good answer cites the document that directly supports the claim. A bad answer either omits citations, cites unrelated sources, overstates the evidence, or adds unsupported API details.

The implementation exposes citations in two places: inline source IDs in the generated answer and a source table in the Gradio UI. This makes each claim traceable to the retrieved Stack Exchange question.

5. Interface

The UI is implemented in Gradio. Users enter a question, choose BM25 or hybrid retrieval, select the number of citations, and optionally enable the HuggingFace generator. The result includes an answer textbox and a retrieved-source table with rank, document ID, title, score, and URL.

```
python3 app.py
```

In Colab, the notebook launches Gradio with `share=True` for a public demo URL.

The interface is intentionally small: the goal is real-time technical Q&A; rather than a complex dashboard. The retrieved-source table is part of the core user experience because it lets the evaluator inspect whether the answer is grounded. For live demos, the recommended flow is to start with BM25 mode to show immediate top-10 retrieval, then switch to hybrid mode after embeddings are built, and finally enable the generator for a cited answer example.

6. Observations and Limitations

BM25 remains a strong baseline because Stack Exchange titles and tags contain concise technical keywords. Dense retrieval is useful when questions are phrased more naturally than the document titles. RRF is a practical fusion choice because it avoids score normalization and tends to preserve strong matches from either retriever.

The main limitation is that the corpus contains only questions, not accepted answers. The RAG layer therefore generates answers from question text and metadata, which is sufficient for retrieval/citation demonstration but weaker than a full answer corpus. A production version should index accepted answers, chunk longer posts, cache embeddings, and add stricter citation verification.

Another limitation is that the current evaluator uses one relevant document per query. Real technical QA often has multiple valid sources. A stronger evaluation set would include graded relevance judgments from multiple annotators, separate dev/test splits, and more diverse query styles. The current evaluation is still useful for the class milestones because it verifies that each retrieval method can recover known target documents and that the metrics pipeline is reproducible.

7. Reproducibility and Submission Notes

The repository includes both script and notebook entry points. `scripts/evaluate.py` is the repeatable command-line evaluator. `notebooks/StackLite_Hybrid_QA_Demo.ipynb` is the Colab-facing walkthrough and contains setup, corpus loading, BM25 top-10 inspection, metrics, dense retrieval, hybrid fusion, RAG generation, and Gradio launch cells. `scripts/make_notebook.py` regenerates the notebook from the project code, which reduces drift between notebook and source files.

The final video file, `video/stacklite_hybrid_qa_demo.mp4`, is 216 seconds long. It summarizes the architecture, metrics, RAG citation format, UI, and deliverables. The companion `docs/video_demo_script.md` provides a live-demo script that team members can use if they record a screen-based version later.

For team work, the implementation can be divided cleanly: one member owns preprocessing and BM25; a second member owns MiniLM embeddings, RRF, and metrics; a third member owns RAG prompting, citation checks, UI, and packaging. The code follows that split without forcing separate notebooks, so the final submission remains easy to run.
